Numerical Methods Project 1

Problem B8: Oceanographic Data Analysis

Students: Dragomir Andrei-Raul, Alexander Hauer, Gheorghita Andrei

Contents

1 Introduction **2**

2 Problem Description and Real-World Context **2**

3 Dataset and Assumptions **2**

4 Mathematical Methods **3**

4.1 Lagrange Interpolation 3
4.2 Newton's Divided Differences 3
4.3 Neville's Algorithm 4
4.4 Cubic Spline Interpolation 4
4.5 Least-Squares Polynomial Fitting 5

5 Manual Interpolation Using Three Consecutive Points **5**

5.1 Lagrange Interpolation 5
5.2 Newton's Divided Differences 6
5.3 Neville's Algorithm 7
5.4 Interpretation of the Manual Result 8

6 Python Implementation Summary **8**

7 Results and Discussion **8**

7.1 Three-Point Interpolation 8
7.2 Polynomial Interpolation on the Full Dataset 9
7.3 Natural Cubic Spline Interpolation 10
7.4 Interpolation Versus Least Squares on Noisy Data 10
7.5 General Comparison of Methods 11

8 Exact Interpolation vs Best-Fit in the Presence of Noise **12**

9 Conclusions **12**

1 Introduction

Interpolation and approximation are important topics in numerical methods because, in real applications, data is usually known only at a finite number of points. From those points, we often want to reconstruct a continuous function, estimate values between measurements, or identify the general trend of the data. These tasks appear in engineering, physics, environmental science, and many other domains.

In this project, the chosen application is oceanographic data analysis. More precisely, we consider temperature measurements taken at different depths in a water column. The main objective is to reconstruct intermediate temperature values and compare several numerical methods: polynomial interpolation, cubic spline interpolation, and least-squares fitting. The project also highlights the difference between exact interpolation and best-fit approximation when the data contains noise.

2 Problem Description and Real-World Context

Oceanographic measurements are often collected at discrete depths because sensors cannot measure every possible point continuously. For example, a probe may record temperature at depths such as 0 m, 50 m, 100 m, and so on. However, in practice, researchers may also want to estimate the temperature at intermediate depths where no direct measurement is available.

This motivates the use of interpolation methods. An exact interpolant constructs a function that passes through every known data point. This is useful when the data is assumed to be precise. On the other hand, in real experiments, measurements may contain noise caused by sensor limitations, water movement, or environmental fluctuations. In that case, a best-fit curve may provide a more realistic overall model than an exact interpolant.

For this reason, the present study compares several methods:

- polynomial interpolation using Lagrange's formula;
- polynomial interpolation using Newton's divided differences;
- Neville's algorithm;
- natural cubic spline interpolation;
- least-squares polynomial fitting.

3 Dataset and Assumptions

The sample oceanographic dataset provided for Problem B8 is shown in Table 1.

Table 1: Depth–temperature dataset used in the project

Depth x (m)	Temperature $T(x)$ (°C)
0	25
50	22
100	20
150	18
200	16

The following assumptions were made:

1. Temperature depends only on depth in this simplified model.
2. The variation of temperature with depth is smooth.
3. The measured values are representative of a vertical temperature profile in ocean water.
4. Real measurements may contain noise, even if the original sample data is clean.

The observed data suggests a decreasing temperature profile as depth increases, which is physically reasonable for a simplified ocean model.

4 Mathematical Methods

4.1 Lagrange Interpolation

Given $n+1$ distinct data points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$, the Lagrange interpolating polynomial is

$$P_n(x) = \sum_{i=0}^{n} y_i L_i(x),$$

where the Lagrange basis polynomials are

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

This method gives an explicit polynomial that passes exactly through all the given points.

4.2 Newton's Divided Differences

Newton interpolation constructs the polynomial in the form

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots,$$

where the coefficients are divided differences:

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i},$$

and higher-order divided differences are obtained recursively by

$$f[x_i, \ldots, x_{i+k}] = \frac{f[x_{i+1}, \ldots, x_{i+k}] - f[x_i, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

This form is convenient because new points can be added without rebuilding the entire polynomial from the beginning.

4.3 Neville's Algorithm

Neville's algorithm is a recursive method for evaluating the interpolating polynomial at a chosen value x. It is based on the recurrence

$$P_{i,i}(x) = y_i,$$

$$P_{i,j}(x) = \frac{(x - x_i)P_{i+1,j}(x) - (x - x_j)P_{i,j-1}(x)}{x_j - x_i}, \qquad i < j.$$

Although Neville's method is mainly used for evaluation rather than symbolic expansion, it is a reliable and structured way to compute interpolated values.

4.4 Cubic Spline Interpolation

A cubic spline is a piecewise polynomial interpolation method. On each interval $[x_i, x_{i+1}]$, the spline is a cubic polynomial

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

The spline is constructed so that:

- it passes through all data points;
- it has continuous first and second derivatives;
- for a natural cubic spline, the second derivative is zero at the endpoints.

Compared to a single high-degree polynomial, cubic splines usually produce smoother and more stable reconstructions.

4.5 Least-Squares Polynomial Fitting

Least-squares fitting does not force the approximation to pass through every data point. Instead, it finds a polynomial $p(x)$ that minimizes the sum of squared residuals:

$$\sum_{i=0}^{n} \left(y_i - p(x_i)\right)^2 .$$

For a polynomial of degree m,

$$p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_m x^m,$$

the coefficients are chosen to best represent the overall trend of the data. This method is especially useful when the data contains measurement noise.

5 Manual Interpolation Using Three Consecutive Points

For the hand-worked part, three consecutive points from the dataset were selected:

$$(50, 22), \qquad (100, 20), \qquad (150, 18).$$

These points are centered in the dataset and are convenient for manual computation.

5.1 Lagrange Interpolation

For the three-point case, the interpolating polynomial has degree at most 2:

$$P_2(x) = 22L_0(x) + 20L_1(x) + 18L_2(x),$$

where

$$L_0(x) = \frac{(x-100)(x-150)}{(50-100)(50-150)} = \frac{(x-100)(x-150)}{5000},$$

$$L_1(x) = \frac{(x-50)(x-150)}{(100-50)(100-150)} = -\frac{(x-50)(x-150)}{2500},$$

$$L_2(x) = \frac{(x-50)(x-100)}{(150-50)(150-100)} = \frac{(x-50)(x-100)}{5000}.$$

Therefore,

$$P_2(x) = 22 \cdot \frac{(x-100)(x-150)}{5000} - 20 \cdot \frac{(x-50)(x-150)}{2500} + 18 \cdot \frac{(x-50)(x-100)}{5000}.$$

Expand the products:

$$(x-100)(x-150) = x^2 - 250x + 15000,$$

$$(x-50)(x-150) = x^2 - 200x + 7500,$$

$$(x-50)(x-100) = x^2 - 150x + 5000.$$

Substituting these expressions gives

$$P_2(x) = \frac{22}{5000}(x^2 - 250x + 15000) - \frac{20}{2500}(x^2 - 200x + 7500) + \frac{18}{5000}(x^2 - 150x + 5000).$$

Combining like terms leads to

$$P_2(x) = 0 \cdot x^2 - 0.04x + 24.$$

Hence the simplified interpolating polynomial is

$$\boxed{P_2(x) = 24 - 0.04x.}$$

Even though a quadratic interpolant was constructed, the coefficient of x^2 is zero because the three points are collinear.

5.2 Newton's Divided Differences

Using the notation

$$x_0 = 50, \quad x_1 = 100, \quad x_2 = 150,$$

$$f(x_0) = 22, \quad f(x_1) = 20, \quad f(x_2) = 18,$$

the divided differences are computed as follows.

First-order divided differences:

$$f[x_0, x_1] = \frac{20-22}{100-50} = \frac{-2}{50} = -0.04,$$

$$f[x_1, x_2] = \frac{18-20}{150-100} = \frac{-2}{50} = -0.04.$$

Second-order divided difference:

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{150-50} = \frac{-0.04-(-0.04)}{100} = 0.$$

The divided-difference table is shown in Table 2.

Table 2: Divided-difference table for the three-point subset

x_i	$f[x_i]$	First divided difference	Second divided difference
50	22	-0.04	0
100	20	-0.04	
150	18		

The Newton interpolating polynomial is

$$P_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1).$$

Substituting the values:

$$P_2(x) = 22 - 0.04(x - 50) + 0 \cdot (x - 50)(x - 100).$$

Thus,

$$P_2(x) = 22 - 0.04x + 2 = 24 - 0.04x,$$

so the final result is again

$$\boxed{P_2(x) = 24 - 0.04x.}$$

5.3 Neville's Algorithm

Neville's algorithm builds the interpolation recursively. For a general evaluation point x, start with

$$P_{0,0}(x) = 22, \qquad P_{1,1}(x) = 20, \qquad P_{2,2}(x) = 18.$$

Then compute

$$P_{0,1}(x) = \frac{(x - 50)P_{1,1}(x) - (x - 100)P_{0,0}(x)}{100 - 50}$$

$$= \frac{20(x - 50) - 22(x - 100)}{50} = \frac{20x - 1000 - 22x + 2200}{50} = \frac{1200 - 2x}{50} = 24 - 0.04x,$$

and

$$P_{1,2}(x) = \frac{(x - 100)P_{2,2}(x) - (x - 150)P_{1,1}(x)}{150 - 100}$$

$$= \frac{18(x - 100) - 20(x - 150)}{50} = \frac{18x - 1800 - 20x + 3000}{50} = \frac{1200 - 2x}{50} = 24 - 0.04x.$$

Finally,

$$P_{0,2}(x) = \frac{(x - 50)P_{1,2}(x) - (x - 150)P_{0,1}(x)}{150 - 50}.$$

Since $P_{1,2}(x) = P_{0,1}(x) = 24 - 0.04x$, the expression simplifies directly to

$$P_{0,2}(x) = 24 - 0.04x.$$

Therefore, Neville's algorithm gives the same interpolating polynomial:

$$\boxed{P_2(x) = 24 - 0.04x.}$$

5.4 Interpretation of the Manual Result

All three methods lead to exactly the same polynomial, as expected. In this case, the result is linear:

$$T(x) = 24 - 0.04x.$$

This means that, on the selected depth interval, the temperature decreases by 0.04 °C per meter, or equivalently by 2 °C every 50 meters.

6 Python Implementation Summary

The numerical methods were implemented in Python. The Lagrange interpolation formula, Newton's divided differences, and Neville's algorithm were coded from scratch in order to verify the hand-computed results for the three-point subset and to extend the interpolation to the full five-point dataset.

In addition, natural cubic spline interpolation was applied on the full dataset in order to obtain a smooth piecewise reconstruction. A least-squares polynomial fit was also computed to study the difference between exact interpolation and best-fit approximation when measurement noise is present.

The program also generated plots for visual comparison between methods. These plots were used in the analysis section of the report.

7 Results and Discussion

7.1 Three-Point Interpolation

Figure 1 shows the interpolation obtained from the three selected consecutive points. Because these three data points are collinear, the degree-2 interpolating polynomial reduces to a straight line.

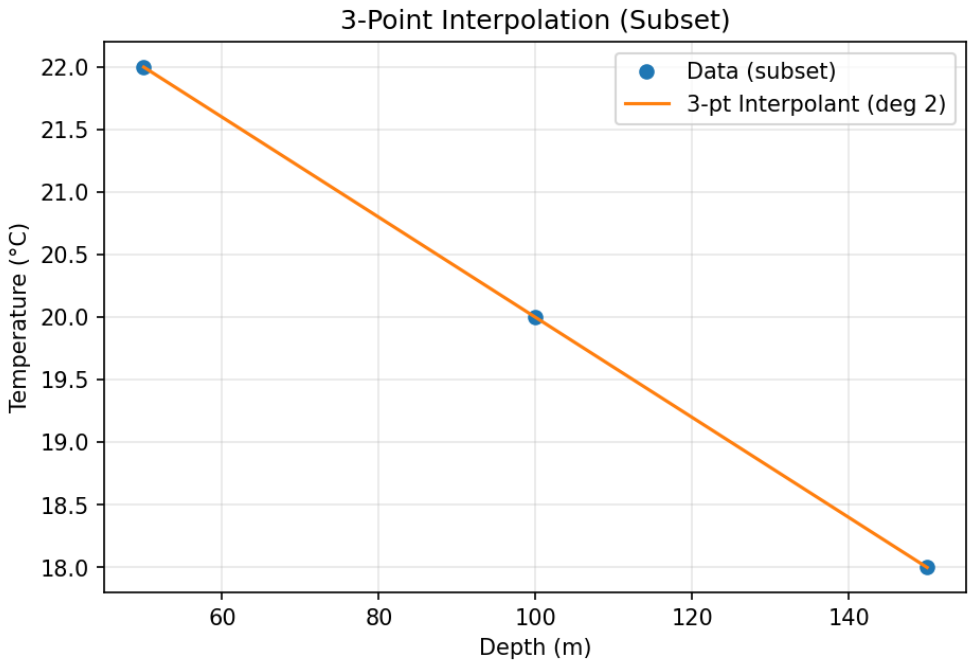

Figure 1: Three-point interpolation on the subset $(50, 22)$, $(100, 20)$, $(150, 18)$.

The graph confirms the manual derivation. Although the interpolation was built as a quadratic polynomial, it behaves exactly like a linear function over this interval.

7.2 Polynomial Interpolation on the Full Dataset

Figure 2 presents polynomial interpolation on the full five-point dataset using both the Lagrange and Newton formulations.

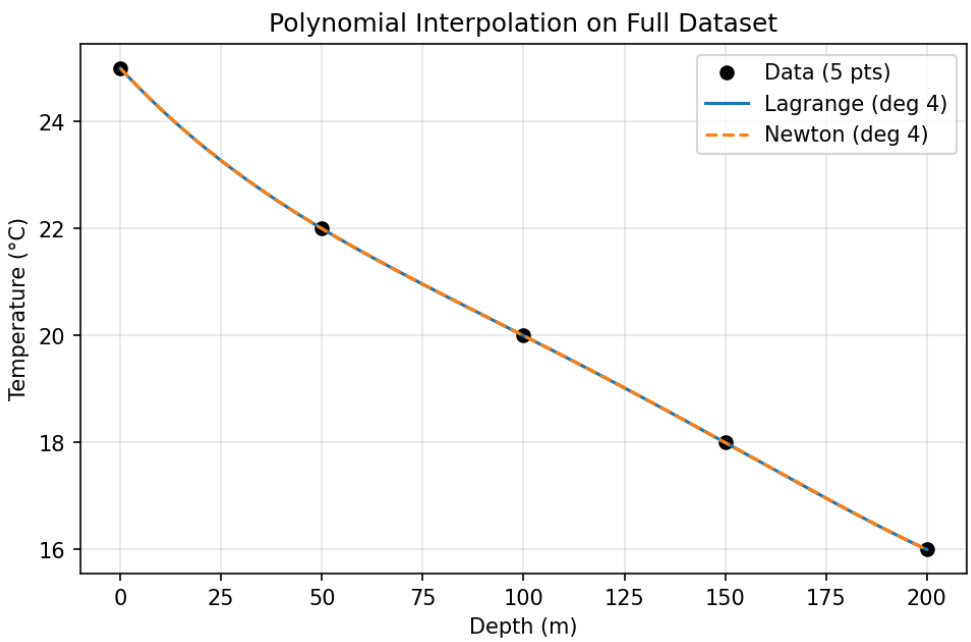

Figure 2: Polynomial interpolation on the full dataset using Lagrange and Newton forms.

From the graph, the two curves overlap visually, which is expected because they represent the same interpolating polynomial written in different forms. The curve passes through all five known data points exactly. In this dataset, the trend is almost linear, so the resulting degree-4 interpolating polynomial does not show strong oscillations.

7.3 Natural Cubic Spline Interpolation

Figure 3 shows the natural cubic spline interpolation on the full dataset.

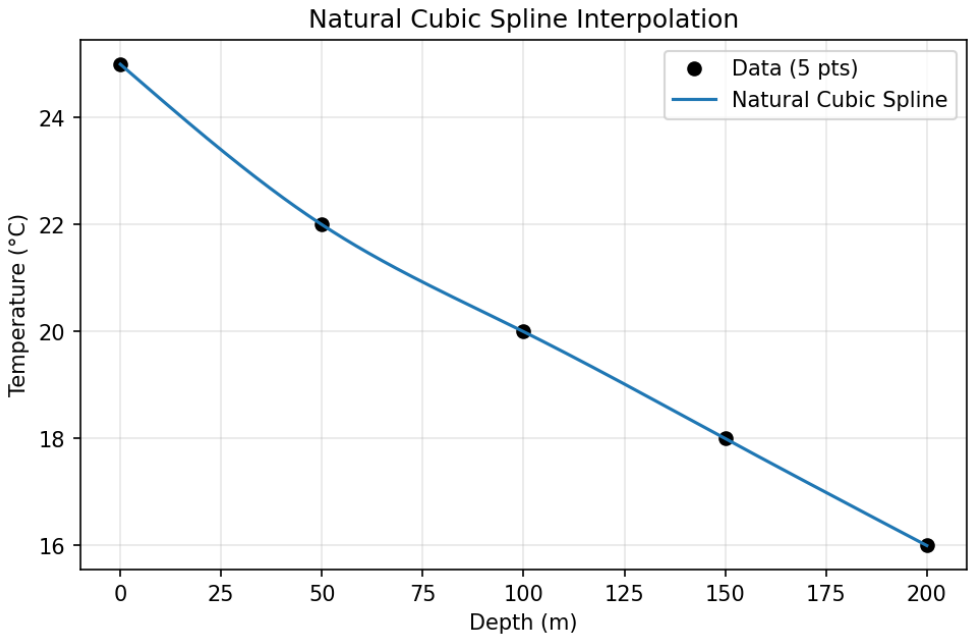

Figure 3: Natural cubic spline interpolation on the full dataset.

The spline also passes through all data points exactly, but unlike the global polynomial interpolant, it is piecewise cubic. This gives a smooth reconstruction with continuous first and second derivatives. For physical data such as ocean temperature profiles, this smoothness is often desirable because abrupt changes are not expected in such a simplified setting.

7.4 Interpolation Versus Least Squares on Noisy Data

To illustrate the influence of measurement noise, a perturbed dataset was considered in the Python experiment. Figure 4 compares the original data, noisy data, an exact interpolant, the cubic spline, and a least-squares polynomial fit.

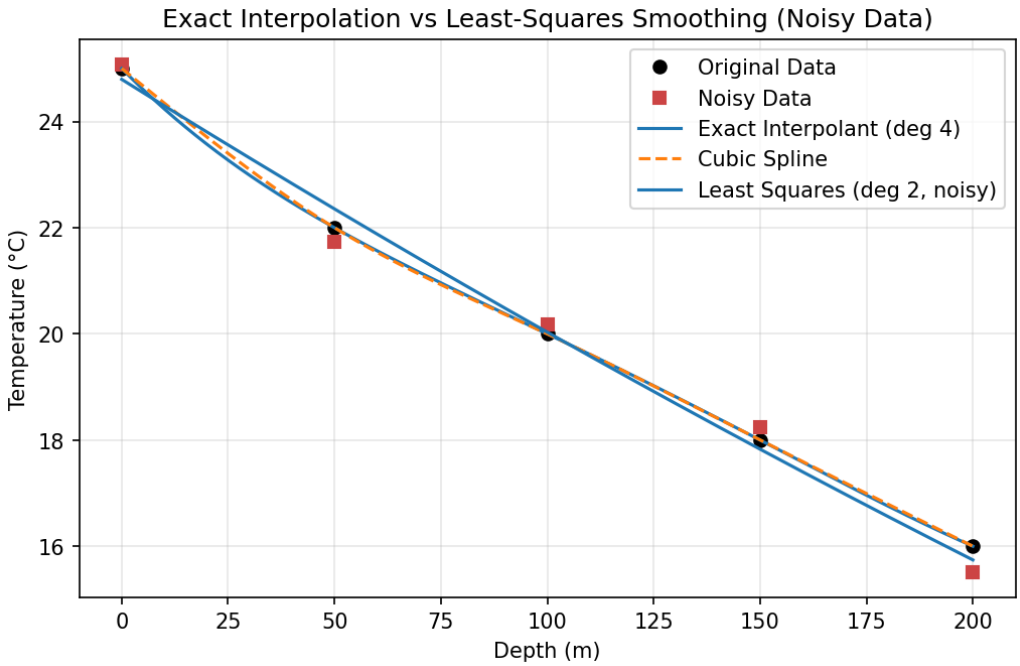

Figure 4: Exact interpolation versus least-squares smoothing for noisy data.

This figure highlights an important conceptual difference:

- the exact interpolating polynomial and the spline attempt to pass through the noisy points;
- the least-squares fit follows the global trend instead of reproducing every fluctuation.

This makes least squares especially useful when the measurements are affected by experimental noise.

7.5 General Comparison of Methods

For the present dataset, the numerical methods lead to consistent results because the underlying trend is simple and smooth. However, they have different strengths:

- **Lagrange interpolation** is direct and theoretically clear, but it becomes less convenient for large datasets.
- **Newton interpolation** is easier to update when new points are added.
- **Neville's algorithm** is practical for evaluating interpolated values at specific points.
- **Cubic spline interpolation** gives a smooth piecewise approximation and is often more stable than a single high-degree polynomial.
- **Least-squares fitting** is preferable when the goal is to capture the overall trend of noisy data rather than pass exactly through each point.

8 Exact Interpolation vs Best-Fit in the Presence of Noise

Exact interpolation assumes that every data point should be matched perfectly. This is appropriate when measurements are exact or when preserving all values is essential. In that case, the interpolating curve is forced to pass through every point.

However, in practical scientific measurements, data is often affected by noise. A sensor can introduce small errors, environmental conditions may vary, and repeated measurements are rarely perfectly consistent. If noisy data is interpolated exactly, the resulting curve may reflect the noise rather than the true physical trend.

Least-squares fitting addresses this issue by minimizing the overall squared error instead of forcing exact agreement at each point. As a result, it usually produces a smoother and more robust description of the underlying phenomenon. For oceanographic temperature data, where the physical variation with depth is generally smooth, a best-fit curve can provide a more realistic model when noise is present.

Therefore, the choice between exact interpolation and approximation depends on the application:

- if all measured values must be reproduced exactly, interpolation is appropriate;
- if the data is noisy and the main objective is trend estimation, least squares is often more suitable.

9 Conclusions

This project studied the numerical reconstruction of ocean temperature data measured at different depths. Three exact interpolation methods were considered for the hand-computed part: Lagrange interpolation, Newton's divided differences, and Neville's algorithm. For the selected three-point subset, all methods produced the same simplified polynomial,

$$T(x) = 24 - 0.04x.$$

The full dataset was then analyzed computationally using polynomial interpolation, natural cubic spline interpolation, and least-squares fitting. The generated plots showed that the original sample data follows a simple decreasing trend with depth. Polynomial interpolation and cubic spline interpolation both pass exactly through the known data points, while least squares is better suited for noisy data because it emphasizes the overall trend.

Among the methods studied, cubic splines are especially attractive for smooth physical profiles because they combine exact interpolation with a smooth piecewise structure. Least squares becomes important when measurement noise is present and a stable trend estimate is required.

A limitation of this project is that the dataset is small and simplified. Real oceanographic data may involve larger datasets, irregular spacing, multiple physical variables, and more complex behavior. Even so, this study provides a clear illustration of how interpolation and approximation methods can be applied in practice.

References

[1] Richard L. Burden and J. Douglas Faires, *Numerical Analysis*, Cengage Learning, standard university edition.

[2] Steven C. Chapra and Raymond P. Canale, *Numerical Methods for Engineers*, McGraw-Hill.

[3] Kendall E. Atkinson, *An Introduction to Numerical Analysis*, John Wiley & Sons.

[4] Alfio Quarteroni, Riccardo Sacco, and Fausto Saleri, *Numerical Mathematics*, Springer.

[5] Carl de Boor, *A Practical Guide to Splines*, Springer.